MP Online Limited

LSPD: Business Requirement Document

Application Name: LSPD (Local Service Provider Directory)

Purpose: Providing Local Services (e.g. Electrician, Plumber, etc.) Throughout MP

Frontend: React

Backend: FastAPI

Database: Postgre sql Server

1. Executive Summary

The MP Local Service Provider Directory is a digital platform designed to function as a modern, location-aware version of traditional directories like Yellow Pages, specifically tailored for Madhya Pradesh.
The primary objective of this system is to bridge the gap between local service providers (e.g., electricians, plumbers, carpenters) and residents by offering a trusted, verified, and location-based discovery platform.
The platform will leverage modern web technologies to provide:

- Seamless user experience via responsive UI
- Accurate geolocation-based search
- Trust-building mechanisms such as verified badges and reviews

2. User Personas

- **2.1 Customer (End User)**

  Profile:

  - Residents of Madhya Pradesh
  - Individuals seeking reliable local services

- Needs:
  - Quick access to nearby service providers
  - Trustworthy and verified professionals
  - Easy comparison via ratings and reviews
  - Map-based navigation

- Pain Points:
  - Difficulty finding reliable local services
  - Lack of trust in unknown providers
  - Time-consuming search process

- **2.2 Service Provider**

  Profile:

  - Local professionals (electricians, plumbers, etc.)
  - Small business owners

  - Needs:
  - Increased visibility in their local area
  - Ability to showcase services and contact details
  - Trust validation through verification badge

  - Pain Points:
  - Limited online presence
  - Difficulty reaching nearby customers
  - Lack of structured promotion channels

- **2.3 Super Admin (MP Online Authority)**

  Profile:

  - Government or authorized administrative body

- Needs:
  - Control over platform integrity
  - Ability to verify providers
  - Monitor and regulate user activity

- Responsibilities:
  - Approve/reject service providers
  - Assign "Verified" badges
  - Manage complaints and reviews

3. Functional Requirements

- **3.1 Customer Features**
  - User registration and authentication
  - Automatic geolocation detection
  - Search services by:
    - Service type
    - District/city within MP
    - Ratings

- View providers on an interactive map
- Access provider details
- **Submit ratings and reviews**

- **3.2 Service Provider Features**
  - Registration and profile creation
  - Add and manage:
    - Service categories
    - Location details
    - Contact information
  - Upload documents for verification
  - Update profile information

- **3.3 Admin Features**
  - Dashboard for system monitoring
  - Approve/reject provider registrations
  - Document verification system
  - Assign and revoke “Verified” badges
  - Manage service categories
  - Monitor and moderate reviews

- **3.4 Map s Geolocation Features**
  - Integration with mapping tools such as Google Maps API or Leaflet
  - Display nearby providers using map markers
  - Interactive UI for viewing provider details
  - Radius-based search (e.g., within 5–10 km)

- **3.5 Rating s Review System**
  - Users can rate providers (1–5 scale)
  - Users can submit textual feedback
  - System calculates average rating
  - Admin moderation capabilities

- **3.6 Verified Badge Logic**
  - Providers upload documents (licenses, ID proof)
  - Admin reviews submitted documents
  - Verified providers receive a visible badge
  - Badge enhances trust and ranking priority

4. Technical Architecture (High-Level)

- **4.1 Frontend**
  - Built using React.js
  - Responsibilities:
    - UI rendering
    - Map visualization
    - API consumption

- **4.2 Backend**
  - Developed using FastAPI (Python)
  - Responsibilities:
    - RESTful API development
    - Authentication C authorization
    - Business logic processing
    - Geolocation queries

- **4.3 Database**
  - PostgreSQL with PostGIS for spatial queries
  - Handles:
    - User data
    - Provider details
    - Reviews
    - Location coordinates

- 4.4 **System Interaction Flow**
  1. User interacts with React frontend
  2. Frontend sends API requests to FastAPI backend
  3. Backend processes logic and queries database
  4. Geolocation queries executed using PostGIS
  5. Results returned and displayed via UI and map

5. Data Requirements

- **Key Entities**

1. Users

- User ID
- Name
- Email
- Role (Customer / Provider / Admin)

2. Service Providers

- Provider ID
- Service Type
- Contact Details
- Verification Status

3. Services

- Service ID
- Category (e.g., electrician, plumber)

4. Reviews

- Review ID
- User ID
- Provider ID
- Rating
- Comments

5. Locations

- Latitude
- Longitude
- Address (District/City in MP)

6. Security s Compliance

- Secure authentication (JWT-based)
- Data encryption for sensitive information
- Role-based access control (RBAC)
- Secure document upload for verification
- Prevention of fake reviews through moderation
- Compliance with basic data privacy standards

7. Success Metrics

The success of the project will be measured based on:

- **User Engagement**
  - Number of registered users
  - Frequency of searches
- **Provider Participation**
  - Number of registered users
  - Frequency of searches
- **Platform Effectiveness**
  - Accuracy of location-based results
  - Average rating reliability
- **System Performance**
  - API response time
  - Map loading speed
- **User Satisfaction**
  - Positive reviews
  - Reduced complaints